云台闭环控制

从零手搓 Step3 教程

2026 FPGA 创新设计赛道（安路）赛题二

视觉方向 • Step3_closed_loop

读者：零基础队员（建议先读完 Step1、Step2 教程）
目标：让云台自己把目标” 追” 回画面中心——从坐标到运动
前置要求：会 Verilog 基础语法、跑过 ModelSim、懂 Step2 的输出
预计学习时间：10～14 天（每天 2～3 小时）

本教程与你手上的代码是什么关系

Step2 的输出是” 目标坐标”；Step3 把它变成云台的**转动**——
误差计算、比例控制、死区、限速限幅、舵机 PWM，一套闭环。

目录

第一部分　从零学习教程 **3**

1　你在造什么 **3**
1.1　一句话 . . . 3
1.2　闭环长什么样 . . . 3
1.3　做完是什么效果 . . . 3

2　阶段 0：反馈控制入门（1～2 天） **3**
2.1　开环 vs 闭环 . . . 4
2.2　一个生活类比：调淋浴水温 . . . 4
2.3　练习 . . . 4

3　阶段 1：舵机与 PWM（2 天） **4**
3.1　舵机怎么听话 . . . 4
3.2　PWM 怎么生成：计数器 + 比较 . . . 4
3.3　上电安全：默认中位 . . . 5
3.4　练习 . . . 5

4　阶段 2：比例控制与定点算术（2～3 天） **5**
4.1　控制律：一帧只做五步 . . . 5
4.2　Q8.8：用移位代替小数 . . . 5
4.3　手算一遍（本阶段练习核心） . . . 6
4.4　练习 . . . 6

5　阶段 3：延迟与稳定性——为什么 Kp 不能大（3 天） **6**
5.1　延迟从哪来 . . . 6
5.2　手推一次振荡（不用任何公式） . . . 6
5.3　增益公式（现场换硬件后要重算的唯一样子） . . . 7
5.4　练习 . . . 7

6　阶段 4：安全设计与状态字（1～2 天） **8**
6.1　四道保险 . . . 8
6.2　丢失目标怎么办 . . . 8
6.3　状态字 O_status . . . 8
6.4　练习 . . . 8

7　阶段 5：跑通闭环测试（1～2 天） **8**
7.1　一键回归 . . . 8
7.2　怎么读结果 . . . 9

第二部分　工程实现详解 **9**

8 工程总览 **9**
8.1 文件清单 . . . 10
8.2 数据流与时钟域 . . . 10
8.3 两条设计原则（本步新增） . . . 10

9 模块详解：gimbal_ctrl **10**
9.1 task 复用：双轴一份逻辑 . . . 10
9.2 五步顺序与实现 . . . 11
9.3 输出同拍锁存” 新值”（一个真 bug） . . . 11
9.4 状态字与丢失策略 . . . 11

10 模块详解：pwm_servo **11**
10.1 µs 到拍的换算 . . . 11
10.2 第二道限幅与上电中位 . . . 12

11 测试平台设计 **12**
11.1 tb_gimbal：10 步进表覆盖全部分支 . . . 12
11.2 tb_pwm：从任务式同步到连续监视器 . . . 12
11.3 tb_closed_loop：相机-舵机模型在环 . . . 13

12 验证结果与振荡事故复盘 **13**
12.1 回归矩阵 . . . 13
12.2 Step3 抓到的真缺陷 . . . 13
12.3 振荡事故复盘（为什么值得单独一节） . . . 13

13 上板挂载说明（给上板组） **14**

14 小结：Step3 的可复用产出 **14**

结语 **15**

总结与下一步 **16**
Step3 学到了什么 . . . 16
下一步学什么 . . . 16

从零学习教程

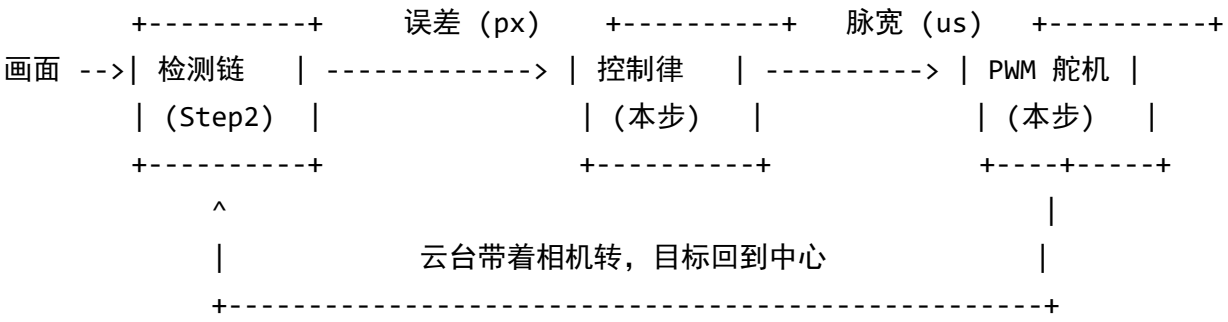
1 你在造什么

1.1 一句话

把”目标在画面哪里”变成”云台往哪转多少”，让镜头自动追着目标走。

Step2 每帧告诉你：目标中心在 `(cx, cy)`。Step3 要回答：舵机脉宽该加多少微秒——左偏 100 像素，脉宽该从 1500 变成多少？

1.2 闭环长什么样



这是一个**反馈闭环**：误差决定动作，动作改变误差。这一节教你写”大脑”（`gimbal_ctrl`，控制律）和”肌肉”（`pwm_servo`，舵机驱动）。

1.3 做完是什么效果

上电后云台在中位静止 → 屏幕角落看到红色目标偏左 → 云台**平滑地**转过去，几秒内目标稳稳停在画面中心小方框里，不再抖动。配合 Step4 的 OSD，画面上还能显示当前状态（LOCK / LOST、坐标、误差）。

学完 Step3 你能做到

- 说清闭环的四个要素：测量、误差、控制、执行
- 手算 Q8.8 定点比例控制的一帧步进
- 解释死区、限速、限幅各防什么事故
- 用增益公式判断一组 Kp 会不会振荡
- 跑通 3 个测试（步进表、PWM 测量、闭环在环收敛）

2 阶段 0：反馈控制入门（1～2 天）

2.1 开环 vs 闭环

两种控制方式

开环：只发命令不看结果。像蒙着眼走路——步子迈出去就收不回来，地面稍有起伏就摔。
闭环：先看结果和目标的差距（误差），再决定下一步动作。误差大就大步走，误差小就小步走，到了就停。

云台跟踪天生就是闭环：目标会动、云台有机械误差、装配有偏差，这些都不用精确知道——只要每帧看一眼” 偏了多少”，闭环自己会修正。

2.2 一个生活类比：调淋浴水温

你想把水温调到 40 度（目标），手拧阀门（执行），皮肤测温（测量）：

- 水太凉 → 往热拧一点；太烫 → 往冷拧一点（比例控制）
- 快到 40 度了 → 只敢轻轻拧（误差小，步子小）
- 在 39～41 度之间 → 不动了（死区，防止来回抖）
- 阀门拧到底了 → 不再硬拧（限幅，行程保护）

新手最常见的直觉错误

” 误差大就猛打”——听起来合理，实际会**振荡**：你猛拧到 40 度，但热水管道有延迟，过两秒温度冲到 45 度；你又猛拧回来，又冲到 35 度……永远停不下来。**闭环里的” 延迟” 决定了你敢用多大的步子**，这是本教程最重要的一个知识点（阶段 3 展开）。

2.3 练习

1. 用” 调水温” 的类比，说出测量、误差、控制、执行分别对应什么
2. 想一个开环控制会失败、闭环能救回来的场景
3. 思考：为什么” 步子大” 在无延迟时很好、有延迟时会振荡？

3 阶段 1：舵机与 PWM（2 天）

3.1 舵机怎么听话

本项目用 SG90/MG90S 级别的位置舵机：三根线（电源、地、信号），信号是一路 50Hz 的 PWM 脉冲，**脉冲宽度**决定转角：

```
0.5ms ----- 0 度
1.0ms ----- 45 度
1.5ms ----- 90 度（中位）
2.0ms ----- 135 度
2.5ms ----- 180 度
周期固定 20ms（50Hz），每个周期只发一个脉冲
```

换算成比例：180 度 / 1000us = **0.18 度/us**。这个数后面算增益要用，先记住。

3.2 PWM 怎么生成：计数器 + 比较

FPGA 生成 PWM 只需要一个计数器和一个比较器：

```
// 50MHz 时钟：20ms = 1,000,000 拍；1us = 50 拍
period_cnt <= (period_cnt >= PERIOD-1) ? 0 : period_cnt + 1;
pwm = (period_cnt < duty_us * 50);   // 前 duty 微秒输出高电平
```

想输出 1500us 脉冲？让 pwm 在计数器前 75000 拍为高。分辨率是 1 拍 = 20ns = 0.02us，比舵机的机械精度高两个数量级，绰绰有余。

为什么”共周期”是硬要求

水平、垂直两个舵机**必须共用同一个周期计数器**。如果各用各的，两个计数器相位慢慢漂移，X 轴脉宽落点和 Y 轴错开，供电电流互相拉扯，舵机抖动。pwm_servo 里一个计数器出两路比较，结构性杜绝。

3.3 上电安全：默认中位

复位后 duty 寄存器就是 1500——云台上电即指向正前方，不会猛甩。这是比赛硬约束”冷启动直接跟踪”的一部分：固化进 bit 流的参数让板子一上电就处于可演示状态。

3.4 练习

1. 手算：想让云台转到 120 度，脉宽多少 us？对应多少拍（50MHz）？
2. 手算：1500us 脉宽 = 多少拍？
3. 思考：如果时钟是 100MHz，1us 等于多少拍？分辨率提升多少？

4 阶段 2：比例控制与定点算术（2～3 天）

4.1 控制律：一帧只做五步

每个”帧拍”（每秒 60 次）执行一遍，以水平轴为例：

```
1) err  = cx - 640                    // 误差：目标偏离中心多少像素
2) s    = (err * KP) >>> 8            // 比例：像素 -> us，Q8.8
3) 死区：|err| <= 25  ->  s = 0        // 目标在中心附近不动
4) 限速：|s| > 100    ->  s = ±100     // 单帧最多加 100us
5) 限幅：duty+s 超出 700~2300 -> 截断   // 行程硬保护
最终：duty = duty_old + s
```

五个步骤的顺序不能乱：先死区（免算）、再限速（防猛）、后限幅（防越界）。

4.2 Q8.8：用移位代替小数

KP 想表达”每像素动 0.078 微秒”这种小数，FPGA 不做浮点。约定：参数里存的是**真值 × 256**（高 8 位整数、低 8 位小数）：

```
KP = 20                    // 参数值（寄存器里存的数）
真值 = 20 / 256 ≈ 0.078 us/px
s = (err * KP) >> 8        // 先乘大数，再右移 8 位除掉 256
```

为什么右移必须用 »>（算术右移）

err 是**有符号**数（目标可以在中心左边）。Python 的 >> 对负数是向下取整，Verilog 的 >>> 配合 signed 才是同一语义。用 >> 或漏 signed，目标在左半边时行为就错——Step2 里 rgb2hsv 已经吃过一次这个亏，这里原样复用纪律。

4.3 手算一遍（本阶段练习核心）

取 KP=20、死区 25、限速 100、限幅 700～2300、当前 duty=1500：

```
例 A：cx = 840      err = 200
      s = (200*20)>>8 = 4000>>8 = 15   duty: 1500 -> 1515
例 B：cx = 660      err = 20    （死区内）
      s = 0                            duty 不动
例 C：cx = 1240     err = 600
      s = (600*20)>>8 = 46             duty: 1500 -> 1546
例 D：cx 在极远处   err = 20000（假设）
      s 本应 1562，限速截到 100        duty: 1500 -> 1600
```

中间量位宽

err(13bit 有符号) × KP(16bit) 的积最多 29 bit——task 里中间量显式声明 signed [28:0]。位宽不够会静默截断，误差大的时候步进值直接变成垃圾。

4.4 练习

1. 手算：err = -150（目标在左），KP=20，duty=1500，新的 duty？
2. 手算：err = 500 时若 KP=90，s 是多少？和 KP=20 差几倍？
3. 用 Python 验证你的答案（注意负数 » 的行为）

5 阶段 3：延迟与稳定性——为什么 Kp 不能大（3 天）

这一阶段是 Step3 的灵魂。控制律本身 20 行就写完，**参数选错，云台当场表演抽搐**——我们在闭环仿真里真的复现了一次（跑飞到 109 度），这里教你提前算出来。

5.1 延迟从哪来

从”目标偏了”到”画面反映云台已转”，要经过：

```
帧 N:   检测链算出 err（1 帧流水）          -> 帧末锁存坐标
帧 N:   控制律算出 step，写 duty             -> 执行
帧 N+1: 舵机转到新角度，画面已变             -> 采集
帧 N+1: 检测链才看到"转完之后"的画面         -> 下一次误差
```

合计约 **2 帧延迟**（1 帧检测 + 1 帧执行）。延迟期间你是在”闭着眼睛”继续输出——这是振荡的根源。

5.2 手推一次振荡（不用任何公式）

假设 Kp 很大，看误差怎么一步步演化（每帧 2 帧延迟，比例全量响应）：

```
帧:   0     1     2     3     4     5     6     7
err:  200   200   -200  -200  +200  +200  -200  -200
      ^^^^^^^^^^ 误差永远在 ±200 之间来回跳 ——等幅振荡！
```

Kp 小一点呢？每次只修正误差的一半：200 → 100 → 50 → 25 → 死区，收敛。**Kp 大到什么程度会开始振荡？**下面用增益算。

5.3 增益公式（现场换硬件后要重算的唯一样子）

三个系数连乘：

$$L = \underbrace{KP/256}_{\text{us/px}} \times \underbrace{0.18}_{\text{度/us}} \times \underbrace{20.6}_{\text{px/度}} \approx KP \times 0.0144 \quad (\text{度/度/帧})$$

再乘延迟放大（2 帧延迟约 ×2），得到**有效增益**，稳定判据：

```
2 × L < 1     即     KP < 256 / (2 × 0.18 × 20.6) ≈ 34
```

代入我们整定时的三组参数验证：

KP	单帧增益 L	有效增益 2L	实测结果
90	1.30	2.60	发散：角度单调跑飞到 109 度
40	0.58	1.16	慢发散振荡（>1，越振越大）
20	**0.29**	**0.58**	**12 帧锁定，无极限环**

*修订注记（2026-10-03）：上表三组实验是初版结构的整定记录（回路按 2 帧延迟）。后续控制节拍修正（tick 改为"新目标包到达拍"，即 T1）把回路延迟降到 1 帧，判据相应放宽为 $L < 1$；**默认 KP=20 不变，实测 settle_frame 由 36 改善到 12**。下面的 ×2 推导保留原样，作为"延迟决定可用增益"的完整示范。*

为什么是"×2"而不是"×1"或"×3"

2 帧延迟的离散比例反馈，每周期误差翻转一次并放大 L 倍，两个帧才完成一个往返——所以是 ×2。若延迟是 1 帧（比如云台响应极快），判据放宽到 L<1；延迟 3 帧则收紧到 L<1/3。**延迟越大，可用增益越小**——这也解释了为什么"检测链要快"。

限速救不了错误的 Kp

有人想"Kp 大了加限速就行"。限速能压住单帧冲击，但压不住符号交替的振荡（每次都 ±100us 来回摆）。先保证 2L<1，再谈限速。

5.4 练习

1. 手算：KP=30 时的 L 和 2L，判断稳不稳
2. 若换一个 FOV 90 度的相机（px/度 = 1280/90 约 14.2），KP 上限变多少？
3. 思考：检测链流水加深 1 帧，KP 上限怎么变？

6 阶段 4：安全设计与状态字（1～2 天）

6.1 四道保险

保险	参数	防什么
死区	DZ=25px	目标在中心附近时云台抖动
限速	SLEW=100us/帧	单帧猛打，机械冲击
软限位	700～2300us	云台撞机械极限（控制律内）
硬限位	500～2500us	软限位失效后的第二道（PWM 内）

6.2 丢失目标怎么办

某帧检测失败（target_valid=0）：**保持当前脉宽不动**，只把 lost_cnt 加一。目标可能只是被遮挡了一瞬，猛回中或猛保持旧方向都可能错——保持是最安全的选择。连续丢失很多帧的应对策略（搜索/回中）留给上层，底层只报告。

6.3 状态字 O_status

8 位状态字，每一位都给 OSD 或调试用：

```
O_status[0] = dz_x    水平在死区内（目标已在中心）
O_status[1] = dz_y    垂直在死区内
O_status[2] = lost    本帧丢失目标
O_status[3] = lim_x   水平限幅命中（打到边界了）
O_status[4] = lim_y   垂直限幅命中
```

track_en 拨码断开时：强制回中位（1500），用于演示复位/安全停车。

6.4 练习

1. 为什么要” 双重限幅”？只有 PWM 里那道行不行？
2. 画出” 丢失目标 3 帧后重新发现目标” 时 duty 的时间线
3. 思考：lost 时保持脉宽，什么情况下这个策略会出问题？

7 阶段 5：跑通闭环测试（1～2 天）

7.1 一键回归

```
cd D:\fpga-ptz-tracker\tools
python run_all.py tb_gimbal tb_pwm tb_closed_loop
```

三个测试各管一层，从里到外：

测试	验证什么	规模
tb_gimbal	控制律 10 步进表（死区/限速/限幅/丢失）	10 帧
tb_pwm	PWM 波形逐拍测量	2 个完整周期
tb_closed_loop	相机-舵机模型在环，60 帧内锁定	全闭环

7.2 怎么读结果

```
--- tb_gimbal ...
    # STEP3 OK duty_x=05E6 duty_y=05D7 lost=0000 dz_x=0 dz_y=1
    ==> PASS          ← 10 步全对（含死区、限速、限幅、丢失保持）

--- tb_pwm ...
    # PERIOD=1000000 pulse_x=75000 pulse_y=75000   ← 上电中位
    ==> PASS          ← 周期与脉宽逐拍精确

--- tb_closed_loop ...
    TB_RESULT: PASS settle_frame=12 final_err=-8
    ==> PASS          ← 12 帧锁定（±25px 死区内保持 10 帧）
```

Step3 学习清单（全部打勾就过关）

- 能说清闭环四要素，以及”每帧一小步”为什么够用
- 能手算一帧步进（含死区/限速/限幅三种截断）
- 能解释 Q8.8 和 `>>>` 的配合
- 能用增益公式判断一组 Kp 的稳定性，说出 KP 上限怎么来的
- 能说出四道保险各防什么、丢失目标的策略
- 跑通 3 个测试并读懂每一行输出

工程实现详解

这一部分写什么

Part I 讲了知识；这一部分把仓库里**真实做出来的 Step3 工程**逐模块讲透：每个参数为什么是这个值、每个赋值为什么用阻塞/非阻塞、三轮 Kp 整定是怎么在闭环仿真里收敛的。对应目录：`03_vision/step3_closed_loop/`——2 个 RTL（209 行）、2 个 TB、3 轮整定记录。

8 工程总览

8.1 文件清单

文件	类型	行数	职责
gimbal_ctrl.v	RTL	140	控制律：死区 + 比例 + 限速 + 限幅，双轴
pwm_servo.v	RTL	69	双路 50Hz PWM，共周期，上电中位
tb/tb_gimbal.v	测试	62	控制律 10 步进表逐字段比对
tb/tb_pwm.v	测试	94	PWM 波形逐拍连续监视器（v2）
sim/gimbal_stim.txt	向量	10	帧输入序列
sim/gimbal_exp.txt	向量	10	金标准期望

8.2 数据流与时钟域

```
Step2 检测链 (74.25MHz 像素域)
   |  目标坐标 + frame_tick (每帧 1 拍)
   v
pulse_sync  (单 bit 脉冲跨域, Step1 模块)
   v
gimbal_ctrl (50MHz PWM 域)  err/step/duty 计算
   |  O_duty_x/y_us (13bit, us)
   v
pwm_servo   (50MHz)          双路 50Hz 波形 -> 舵机
```

两个时钟域：像素域 74.25MHz 跑检测，PWM 域 50MHz 跑控制与波形。跨域只传两样东西：**帧拍脉冲**（pulse_sync，单 bit）和 **目标坐标**（cdc_4phase，多 bit）。50MHz 是刻意选择的：1us 恰好 50 拍，μs 到拍的换算纯乘法无余数。

8.3 两条设计原则（本步新增）

Step3 的方法论

1. 无符号纪律：误差、乘积、累加一律显式 `$signed`；task 内中间量 29bit。整步最阴的 bug 全部出在符号上。

2. 参数即固化：所有控制参数是 RTL parameter——综合进 bit 流，上电即生效，满足”冷启动直接跟踪”的比赛硬约束。

9 模块详解：gimbal_ctrl

9.1 task 复用：双轴一份逻辑

水平、垂直两轴的控制律完全同构，只有参数不同。RTL 用一个 task `calc_axis` 承载全部算术，时序块里调两次：

```
task calc_axis(input signed [12:0] err, input [15:0] kp, ...);
    prod = err * $signed({1'b0, kp});   // 29bit 有符号积
    s    = prod >>> 8;                  // Q8.8 -> us, floor
    ...
endtask
```

注意 $signed({1'b0, kp})：位拼接默认产生无符号数，不包一层 $signed 的话整个乘法退化成无符号，负误差直接回绕——这正是闭环发散事故的模型侧根因（详见振荡复盘一节）。

9.2 五步顺序与实现

```
dz_hit = (err >= -dz) && (err <= dz);      // 1) 死区判定
if (dz_hit) s = 0;                          //    命中则步进清零
if (s >  SLEW_P)  s =  SLEW_P;              // 2) 限速（双向）
else if (s < -SLEW_P) s = -SLEW_P;
nx = $signed({1'b0, duty_in}) + s;          // 3) 累加
if (nx > SMAX_P) begin duty_out = SMAX_P; lim_hit = 1; end
else if (nx < SMIN_P) begin duty_out = SMIN_P; lim_hit = 1; end
```

9.3 输出同拍锁存”新值”（一个真 bug）

初版输出寄存写成 O_duty_x_us <= duty_x——而 duty_x 在同一拍才被赋成新值，非阻塞赋值读的是**旧值**，输出滞后一帧。tb_gimbal 的步进表第一帧就对不上。修复：

```
O_duty_x_us <= (I_track_en && I_target_valid) ? dnx_x : duty_x;
//                                ^^^^^ 本拍刚算出的新值
```

非阻塞赋值的”读旧写新”

非阻塞赋值 RHS 立即求值、LHS 帧末更新。同一个 always 块里”先更新 duty_x，再输出 duty_x”，读到的是更新前的值。要么像本工程一样输出 dnx_x（新值），要么想清楚就是要旧值。

9.4 状态字与丢失策略

```
O_status <= {2'b00, limy, limx, lost, dzy, dzx};   // 一拍拼装
// lost 帧: duty 保持不动, O_lost_cnt 饱和累加
// track_en=0: duty 强制回中位, 状态清零
```

一个迭代 1 抓出的细节：无效帧分支里 dzx/limx 清零用非阻塞，同拍组装的 O_status 采样到**旧值**——状态字残留上一帧的死区位。修复：该分支的清零改**阻塞赋值**（组装在同一个顺序块里，先清后拼）。这是”阻塞/非阻塞选择不是教条、而是看数据依赖”的实例。

10 模块详解：pwm_servo

10.1 µs 到拍的换算

```
localparam TICKS_PER_US = 50;                 // 50MHz / 1e6
wire [19:0] pulse_x = duty_x * TICKS_PER_US; // 1500us -> 75000 拍
```

20bit 计数器数到 1,000,000（50Hz 周期）回零，两路输出 pwm = (period_cnt < pulse)——**一个计数器、两个比较器**，共周期是结构保证的。

10.2 第二道限幅与上电中位

```
wire [12:0] dx = (I_duty_x_us > 13'd2500) ? 13'd2500 :
                 (I_duty_x_us < 13'd500)  ? 13'd500  : I_duty_x_us;
// 复位值: duty_x <= 13'd1500    ——上电即中位
```

控制律已经限在 700～2300，这里再限 500～2500——两道保险的区间故意不同：如果上层逻辑跑飞给出荒谬值，第二道还能兜住机械极限。

11 测试平台设计

11.1 tb_gimbal：10 步进表覆盖全部分支

金标准 `gen_tb_gimbal()` 构造 10 帧输入序列，**刻意覆盖每种截断**：

帧	输入	考察点
0-1	cx=840 (err=200)	常规比例步进（1500 到 1515）
2	cx=900 (err=260)	累加继续
3	cx=700 (err=60)	小误差小步进
4	cx=641 (err=1)	**死区命中，duty 不动**
5-6	valid=0	**丢失保持 + lost_cnt 累加**
7	cx=100 (err=-540)	负方向 + 大步进
8	cx=640 (err=0)	死区
9	cx=660 (err=20)	死区

期望文件每行 `{dx:03X}{dy:03X}{lost:04X}{dzx:X}{dzy:X}`，TB 逐字段比对——duty、lost 计数、两个死区标志**都要对**。第一条期望 `5EB5DC000001` 手工可验：dx=0x5EB=1515（err=200 步进 15）、dy=0x5DC=1500（cy=360 正中死区）、dz_y=1。

11.2 tb_pwm：从任务式同步到连续监视器

第一版 TB 用任务式 `@(posedge tick)` 等待周期起点再测量——实测 `len=1`：TB 的等待事件和 DUT 的置位**同拍竞态**，采样点漂移。v2 重写为**连续监视器**：

```
// 自由运行计数器 + tick 锁存，没有任何 @ 事件同步
always @(posedge clk) begin
    if (pwm) pulse_len <= pulse_len + 1;
    if (tick) begin
        report(frame_cnt, pulse_len, period_len);
        pulse_len <= 0;
    end
    period_len <= (tick) ? 1 : period_len + 1;
end
```

锁存窗口**包含 tick 拍本身**（v1.1 曾差一拍测得 999999），换挡激励避开过渡周期。v2 测得：上电中位 75000/75000/1000000 逐拍精确，换挡后 60000（1200us）/100000（2000us）精确。

11.3 tb_closed_loop：相机-舵机模型在环

真正的验收是把整条链闭环跑起来。TB 内建行为级相机模型（参数都有实物依据）：

参数	取值	依据
光学放大	20 px/度	62.2 度 FOV @ 1280px
舵机增益	0.18 度/us	180 度 / 1000us 行程
机械速率限幅	1.5 度/帧	约 90 度/s
系统延迟	1 帧	检测链流水；控制节拍 = 新目标包到达拍（T1 修正前为 2 帧）
初始指向 theta0	30 度	与目标差 10 度

模型每个 PWM 周期算 `theta_cmd = 30 + (duty-1500)×0.18`，带速率限幅逼近；TPG 方块按 `x = 640 + (40-theta)×20` 实时移动——**相机真的在”看”云台转完的画面**。验收标准：60 帧内误差进入 ±25px 死区并保持 10 帧。

12 验证结果与振荡事故复盘

12.1 回归矩阵

测试	比对粒度	结果
tb_gimbal	10 步进表逐字段（duty/lost/dz 标志）	PASS
tb_pwm	波形逐拍（脉宽/周期）	PASS
tb_closed_loop	在环收敛（settle_frame）	PASS：12 帧锁定

12.2 Step3 抓到的真缺陷

编号	现象	根因	类型
G1	输出滞后一帧	输出寄存同拍读旧值（非阻塞语义）	RTL
G2	状态字残留	无效帧清零用非阻塞，同拍拼装读旧值	RTL
G3	测得脉宽 len=1	任务式 @ 同步与 DUT 置位同拍竞态	TB
G4	周期 999999	计数窗口不含 tick 拍	TB
G5	闭环跑飞 109 度	模型无符号减法回绕 + 回路增益超标	模型 + 参数

12.3 振荡事故复盘（为什么值得单独一节）

整定第一轮 Kp=90/70，闭环仿真 60 帧里角度**单调跑飞到 109 度**，duty 打满限幅。复盘出两个叠加的问题：

事故 1：TB 模型的无符号减法

相机模型算误差时用了无符号减法：`(40 - theta)` 在 theta 超过 40 时回绕成巨大正数，模型认为目标突然飞到另一边——闭环被喂了假误差。修复：全链路 `$signed`。教训：**行为模型也要遵守无符号纪律**，它错的时候 RTL 怎么对都没用。

事故 2：回路增益超标。Kp=90 时单帧增益 L=90×0.0144=1.30，两帧延迟放大后 2L=2.60——每往返误差翻 2.6 倍，单调发散。第二轮 Kp=40（2L=1.16）仍是慢发散；第三轮 Kp=20/18（2L=0.58）进入稳定收敛。**闭环专项修正（T1）**：控制节拍改为" 新目标包到达拍"，回路延迟由 2 帧降到 1 帧（判据放宽为 L<1），实测 **settle_frame 由 36 改善到 12**，RTL 默认参数同步更新。

给现场整定者的操作清单（完整版见 IF-SERVO-PWM.md）

1）先验极性：给固定小误差，云台必须**朝减小误差**的方向动；反了改 `INVERT_X/Y` 参数，不要倒接电机线。
2）从保守 Kp 起步（按增益公式算出上限，取一半）。
3）逐步加大直到出现轻微振铃，回退 30%。
4）SLEW 从 100 起步：只影响" 最多多快"，不影响稳定性判据。
5）改完参数重新综合固化——参数是 RTL parameter，不烧不生效。

13 上板挂载说明（给上板组）

项目	内容
时钟	50MHz PWM 域（官方例程已有该域，直接挂）
跨域输入	目标坐标（cdc_4phase）；控制节拍本地生成 = 新目标包到达拍（T1 后不再用 pulse_sync 传 tic
输出	O_pwm_x/y 接舵机信号线；O_duty_dbg 供 ILA 观测
按键/拨码	track_en 接拨码（0 = 回中位，演示/安全用）
供电	舵机独立 5V 供电，**与 FPGA 共地不共电**
参数	KP/DZ/SLEW/限幅全部是 parameter，现场整定后重编译固化

上板三步：
1. 两个 RTL 加入 TD 工程，实例化 gimbal_ctrl + pwm_servo；
2. 把 Step2 的 `O_frame_tick`/坐标经跨域模块接入；
3. 先不接舵机，用 ILA 看 O_pwm 波形确认周期/脉宽，再带载。

和 Step2 的分工

Step2 只负责" 看"，输出坐标和帧拍；Step3 只负责" 动"，输入坐标和帧拍。两者通过两个跨域信号解耦——检测链改坏了不影响舵机安全（丢失保持），舵机故障也不阻塞检测。**接口即边界**，约定见 `06_interfaces`。

14 小结：Step3 的可复用产出

- **2 个可综合 RTL 模块**（209 行），控制律与波形生成解耦；

- **一套增益设计方法**：L = KP×0.0144，按回路延迟选判据（初版 2 帧延迟用 2L<1；T1 修正后延迟 1 帧，判据放宽为 L<1），换相机/舵机后重算即可；
- **一个在环验证范式**：行为级相机模型 + TPG 方块跟随，无硬件就能整定参数；
- **三份整定记录**：发散/慢发散/锁定的完整过程，是答辩时”工程严谨性”的最好素材。

下一步（Step4 预告）

坐标有了、云台动起来了，但屏幕上还看不到任何”系统在干活”的痕迹。Step4 把结果画回画面：状态文字、目标框、跨域目标包、整链联调——评委会看到的画面，从这里来。

结语

总结与下一步

Step3 学到了什么

- 反馈闭环的思想：测量、误差、控制、执行
- 舵机 PWM 规格（50Hz、0.5～2.5ms、0.18 度/us）与共周期生成
- 控制律五步：误差、Q8.8 比例、死区、限速、限幅
- 稳定性判据：增益公式与 2 帧延迟的 ×2 放大
- 安全设计：四道保险、丢失保持、状态字、上电中位
- 参数整定方法论：先极性、后增益、在环仿真验证

下一步学什么

- **Step4 OSD 与系统联调**：把检测结果画回画面
  - 字库 ROM 与文字叠加（状态行直接显示 LOCK/坐标）
  - 68bit 目标包跨域（视觉域 → 显示域）
  - vision_top 整链集成与全帧逐位比对
- 对应文档：`03_vision/step4_osd_integration/`
- 对应金标准：`tools/golden_model.py::gimbal_ctrl_step`